1. Freeze the scope

Use this as your project statement:

PhishShield: An Explainable ML-Based Phishing URL Detection and Cybersecurity Awareness Platform

Goal: analyze a URL, estimate phishing risk, explain the contributing signals, store scan history, visualize risks, and teach users how to verify suspicious links.

Do not promise real-time internet-wide threat intelligence or perfect detection. Frame it as an **ML-based risk-assessment tool**.

2. Build the ML model first

Use the UCI phishing dataset as the initial benchmark.

- Clean and inspect the features.
- Train Logistic Regression, Decision Tree, Random Forest, and XGBoost.
- Compare F1-score, precision, recall, ROC-AUC, and confusion matrices.
- Select the strongest model.
- Save it as a .joblib/.pkl model.

The key technical decision: initially use only features that your app can calculate safely from a pasted URL:

- URL length
- number of dots/subdomains
- hyphens, @, IP address usage
- HTTPS
- suspicious words/tokens
- number of digits/special characters
- path/query length

Later, add optional lookup-based signals such as domain age or SSL metadata.

3. Make the prediction explainable

This is the part that upgrades it from a basic classifier.

For every result, return:

- predicted class: Safe / Suspicious / High Risk
- confidence/risk score
- top factors that raised or lowered the risk
- plain-language advice

Example:

High risk: 86%
The URL uses an IP address, has multiple subdomains, and contains credential-related terms. Do not enter passwords or payment details; navigate to the organization's official site manually.

Use SHAP or feature importance for the technical explanation, then map those results to understandable text.

4. Build the backend API

Use **FastAPI + Python**.

Core endpoints:

- POST /scan — accepts a URL, extracts features, returns prediction/explanation
- GET /history — scan history
- GET /analytics — aggregate counts, common phishing signals, trend data
- GET /model-metrics — model-performance data for the Model Insights page

Use **SQLite** initially. It is completely adequate for a final project.

5. Build a multi-page frontend

Use React + Tailwind + Recharts, or Streamlit if you need to finish faster.

Pages:

1. **Home / URL Scanner**
   Paste URL, scan, see the result.
2. **Scan Result**
   Risk gauge, confidence, parsed URL, warning signals, safety guidance.
3. **History**
   Searchable list of earlier scans; filter by risk and date.

4. **Threat Analytics**
   Total scans, high-risk percentage, trends, most frequent suspicious patterns.
5. **Model Insights**
   Model comparison, confusion matrix, ROC curve, feature importance. This is especially useful in viva.
6. **Learn & Protect**
   Examples of phishing techniques, red flags, reporting/help guidance.

6. Add one optional "final-project differentiator"

Choose only one after the core system works:

- **Browser extension:** scan the active tab's URL with one click.
- **Screenshot/page visual analysis:** detect suspicious webpage visual patterns.
- **User feedback loop:** users mark predictions as correct/incorrect; save feedback for future retraining.
- **QR-code scanner:** upload a QR code, extract its URL, and scan it.

My vote: browser extension, because it looks impressive but reuses your existing /scan API.